



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ _____ Фундаментальные науки

КАФЕДРА _____ Прикладная математика

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ
НА ТЕМУ:

Задача коммивояжера

Студент _____
ФН2-51Б
(Группа)

(Подпись, дата)

А. С. Джагарян
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

И. К. Марчевский
(И. О. Фамилия)

2024 г.

Оглавление

1. Постановка задачи	3
2. Метод ветвей и границ	3
2.1. Общие сведения	3
2.2. Наивный алгоритма	4
2.3. Построение дерева через ребра	5
3. Tabu Search	13
3.1. Общие сведения	13
3.2. Алгоритм локального поиска	16
3.3. Алгоритм поиск с запретами	17
4. Особенности реализации на языке C++	21
4.1. Метод ветвей и границ	21
4.2. Поиск с запретами	21
5. Особенности реализации на языке C++	22
5.1. Метод ветвей и границ	22
5.2. Поиск с запретами	22
Список использованных источников	23

1. Постановка задачи

Дан набор из n вершин и набор ребер, их соединяющих. Каждое ребро является направленным и характеризуется весом — положительным числом — стоимостью движения из его начала в конец. Для любой пары вершин существует два ребра, их соединяющих (в прямом и обратном направлении). Требуется найти маршрут минимальной стоимости, проходящий через все вершины ровно по одному разу и возвращающийся в первую вершину.

Входные данные: количество вершин и матрица весов ребер, их соединяющих (в общем случае несимметричная).

Выходные данные: номера вершин, образующих маршрут.

Определение 1 $G(n)$ — множество всех последовательностей, которые можно получить из последовательности $1, 2 \dots n$, 1 путем перестановки любых элементов находящиеся со 2ой до n ой позиции.

Введем функцию $t = f(x)$, где $x \in G(n)$, а $f(x)$ — стоимость соответствующего маршрута x .

Задачу коммивояжера можно сформулировать так: Найти $x_0 : t = f(x_0) \rightarrow \min$

2. Метод ветвей и границ

2.1. Общие сведения

Метод ветвей и границ — это алгоритм для решения задач дискретной оптимизации. Данный метод представляет собой полный перебор с отбрасыванием решений, которые явно не могут быть оптимальными, что позволяет значительно сократить число рассматриваемых вариантов.

Определение 2 Верхней границей $\bar{t}$ называется стоимость оптимального пути всех элементов $x \in G(n)$ рассмотренных до текущего момента времени. $(\exists x_0 \in G(n) : \bar{t} = f(x_0) \geq t_{\min}, \text{ где } t_{\min} - \text{оптимальная стоимость на всем множестве } G(n))$

Определение 3 Нижней границей $\underline{t}$ называется стоимость такая, что не существует решения в данной под-области с меньшей стоимостью чем $\underline{t}$. $(\forall \text{ след шаге текущая стоимость маршрута будет } \geq \underline{t})$

Использование метода ветвей и границ состоит из 3ех этапов: ветвление, оценка границ, отсечение.

1. Ветвление: область определения разбивается определенным образом на под-области.
2. Оценка границ: для каждого узла дерева вычисляется нижняя и верхняя граница функции $t = f(x)$.
3. Отсечение: если рассматриваемая под-область не содержит оптимальное решение т.е. $\underline{t} \geq \bar{t}$, то дальше рассматривать данную под-область не имеет смысла и ее можно отсечь.

Данный процесс можно повторять до тех пор пока все множество определения целевой функции не исчерпается.

2.2. Наивный алгоритма

Наивный алгоритм – алгоритм при котором процесс ветвления происходит по вершинам городов.

Узлы этого дерева представляют собой вершины городов с соответствующими номерами, а также содержат информацию о верхней и нижней границах. На каждом уровне дерево ветвится во все возможные непосещенные города.

Например пусть количество городов $n = 4$. Тогда ветвление данной задачи можно представить деревом изображенным на рис. 1

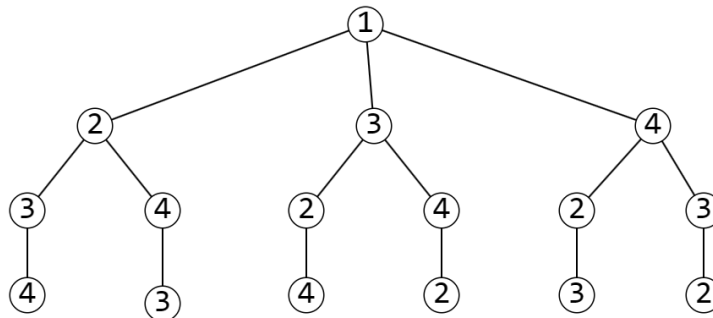


Рис. 1. Дерево

Проходим в глубину по дереву, для каждой вершины считаем верхнюю и нижнюю границу, если нижняя граница оказалась больше чем верхняя, то текущую ветку дерева можно отбрасывать, поскольку оптимального решения там нет.

Ниже приведен псевдокод Наивного алгоритма.

NaiveAlgorithm(size, AdjacencyMatrix)

- 1: Сохраняем текущую вершину в текущий путь
- 2: $\underline{t} += AdjacencyMatrix[ParentVertex][CurrVertex]$

```

3: if  $\underline{t} \geq \bar{t}$  then
4:   return
5: end if
6: if Все города пройдены and  $\bar{t} \geq \underline{t} + \text{AdjacencyMatrix}[\text{CurrVertex}][0]$  then
7:    $\bar{t} = \underline{t} + \text{AdjacencyMatrix}[\text{CurrVertex}][0]$ 
8:   Обновляем лучший путь на текущий
9: else if Все города пройдены then
10:  return
11: else
12:  for child in Оставшиеся города do
13:    Удаляем child из оставшихся городов
14:    NaiveAlgorithm(size, AdjacencyMatrix)
15:  end for
16: end if

```

При таком ветвлении возникает проблема: размерности нижней и верхней границ различны. Нижняя граница в каждом узле дерева оценивает длину не всего пути, а только его часть (от корня до текущего узла). Верхняя граница оценивает длину всего пути целиком. Если расстояния между городами значительно различаются, ненужные ветви будут отбрасываться. Однако, если расстояния между городами будут примерно одинаковыми, значительного отбрасывания ветвей не произойдет; ветви могут отбрасываться только при приближении к листьям дерева. Поэтому отбрасывание будет незначительным. Чтобы сделать размерности верхней и нижней границ одинаковыми, необходимо изменить способ построения дерева.

2.3. Построение дерева через ребра

Проблема размерности

Под ребром будем понимать путь, соединяющий два города. Чтобы сделать размерности верхней и нижней границ одинаковыми, дерево следует строить по ребрам. То есть, каждый узел должен содержать информацию о том, по каким ребрам мы точно идем, а какие ребра являются запретными. На каждом уровне дерево определяет, по какому ребру лучше ветвиться. Влево, если ребро выбирается, и вправо, если оно исключается.

Будем повышать размерность нижней границы до размерности верхней границы, то есть нижняя граница будет описывать наименьшую возможную длину всего пути при условии, что мы находимся в соответствующей вершине дерева.

Решение проблемы размерности

Для этого заметим, что коммивояжер заходит и выходит из каждого города ровно один раз. Это означает, что из города i нам необходимо куда-то выехать, и, куда бы мы ни направлялись, нужно потратить как минимум значение, равное минимальному элементу в i -ой строке. Со въездом аналогично. Тогда нижняя граница может быть оценена следующим образом:

- 1) Находим минимум в каждой строке
- 2) Вычитаем из каждого элемента строки соответствующий минимум
- 3) Находим минимум в каждом столбце
- 4) Вычитаем из каждого элемента столбца соответствующий минимум.
- 5) В корне дерева $\underline{t}$ = сумма всех минимумов

В дальнейшем данную процедуру будем называть редуцированием строк и столбцов. Ниже приведен псевдокод редуцирования строк. AllowRows – это строки, которые еще не были заблокированы алгоритмом, AllowColumns – это столбцы, которые еще не были заблокированы алгоритмом. Аналогично можно реализовать ReductionRowsColumns, ReductionRowsAndColumns.

ReductionRows(size, AdjacencyMatrix, AllowRows, AllowColumns)

```

1: minel =  $\infty$ 
2: suma = 0
3: for i in AllowRows do
4:   minel =  $\infty$ 
5:   for j in AllowColumns do
6:     minel = min(minel, AdjacencyMatrix[i][j])
7:   end for
8:   for j in AllowColumns do
9:     AdjacencyMatrix[i][j] – = minel
10:  end for
11:  suma + = minel
12: end for
13: return suma

```

Изменение матрицы смежности при ветвлении

Рассмотрим, что произойдет с матрицей смежности, если мы выберем ребро $i \rightarrow j$. После выбора ребра $i \rightarrow j$ необходимо заблокировать i -ую строку и j -ый столбец, так как из города i мы поедem только в город j , и в другие города из i мы уже поехать не сможем. Аналогично, въехать в город j можно будет только из города i , то есть

из любого другого города въехать в город j будет невозможно.

Также, после взятия ребра $i \rightarrow j$, нужно заблокировать те ребра, которые приводят к преждевременному заиклииванию пути, чтобы при следующем выборе ребра мы не могли выбрать то ребро, которое заиклиивает путь.

Если мы не выбираем ребро, то есть запрещаем включение ребра $i \rightarrow j$ в наш путь, тогда в матрице смежности нужно просто заблокировать соответствующую ячейку.

Таким образом, при выборе ребра происходит уменьшение размерности матрицы смежности, чего не происходит при блокировке ребра.

Выбор ребра по которому дерево будет ветвится

Для достижения максимального количества отброшенных ветвей дерева необходимо, чтобы нижняя граница была больше верхней. Это означает, что при выборе ребра следует стремиться к максимальному увеличению нижней границы.

Поскольку выбор ребра приводит к уменьшению размерности матрицы смежности, важно обеспечить отбрасывание именно правых ветвей дерева, так как они сохраняют размерность матрицы смежности.

Таким образом, задача состоит в том, чтобы найти ребро, блокировка которого приведёт к наибольшему увеличению нижней границы. Стоит отметить, что после редукции строк в каждой строке и столбце матрицы смежности будет присутствовать хотя бы один нулевой элемент.

Предположим, что заблокировано ребро $i \rightarrow j$. После блокировки коммивояжёр должен покинуть город i и направиться в любой город, кроме j -го. Однако независимо от выбранного направления, он будет вынужден потратить $\min_{k=1\dots n, k \neq j} (AdjacencyMatrix[i][k])$.

Аналогично, при въезде в j -ый город коммивояжёру необходимо выбрать направление из любого города, за исключением i . В любом случае, его затраты не будут меньше чем $\min_{k=1\dots n, k \neq i} (AdjacencyMatrix[k][j])$.

Таким образом нижняя граница вырастет

$$\underline{t} + = \min_{k=1\dots n, k \neq j} (AdjacencyMatrix[i][k]) + \min_{k=1\dots n, k \neq i} (AdjacencyMatrix[k][j])$$

В случае выбора ребра в качестве разбиения дерева, которое имеет ненулевой вес, нижняя граница не увеличится, поскольку в каждой строке и столбце матрицы смежности присутствует хотя бы один нулевой элемент. Следовательно, целесообразно

рассматривать только ребра с нулевым весом. Среди таких ребер необходимо выбрать то, для которого выражение

$$\min_{k=1..n, k \neq j} (AdjacencyMatrix[i][k]) + \min_{k=1..n, k \neq i} (AdjacencyMatrix[k][j]) \rightarrow \max \quad (1)$$

Ниже приведен псевдокод процедуры, которая ищет величины входящие в формулу (1) для заданного ребра. *Coordinate* – координаты того ребра, которое хотим блокировать, *AllowRows* – это строки, которые еще не были заблокированы алгоритмом, *AllowColomns* – это столбцы, которые еще не были заблокированы алгоритмом, *MinXel* – минимальный элемент в строке *Coordinates.x*, не считая самого элемента, *MinYel* – минимальный элемент в столбце *Coordinates.y*, не считая самого элемента.

PenaltySearch(size, *AdjacencyMatrix*, *Coordinates*, *AllowRows*, *AllowColumns*)

```

1: MinXel = ∞
2: MinYel = ∞
3: for i in AllowRows do
4:   MinYel = min (MinYel, AdjacencyMatrix[i][Coortrdinates.y])
5: end for
6: for j in AllowColomns do
7:   MinXel = min (MinXel, AdjacencyMatrix[Coortrdinates.x][j])
8: end for
9: return {MinXel, MinYel}

```

Ниже приведен псевдокод процедуры, которая позволяет найти оптимальное ребро для ветвления дерева с целью максимального увеличения нижней границы.

BestEdjeSearch(size, *AdjacencyMatrix*, *AllowRows*, *AllowColumns*)

```

1: maxPenalty = -∞
2: BestEdjeCoordinates = {0, 0}
3: for i in AllowRows do
4:   for j in AllowColomns do
5:     if AdjacencyMatrix[i][j]==0 then CurrPenalty = PenaltySearch(size, AdjacencyMatrix,
       {i, j}, AllowRows, AllowColumns)
6:       if CurrPenalty.x + CurrPenalty.y > maxPenalty then
7:         maxPenalty = CurrPenalty.x + CurrPenalty.y
8:         BestEdjeCoordinates.x = i
9:         BestEdjeCoordinates.y = j
10:      end if
11:   end if

```



```

12:   end for
13: end for
14: return BestEdgeCoordinates

```

Проблема зацикливания пути Рассмотрим следующую ситуацию. Предположим, что коммивояжер выбрал ребро $i \rightarrow j$. На следующей итерации оказалось, что ребро $j \rightarrow i$ является наиболее выгодным. Однако если коммивояжер пойдет по этому ребру, он окажется в одном и том же городе дважды, что противоречит условию задачи.

Таким образом, после выбора ребра необходимо блокировать те ребра, которые на следующей итерации могут привести к преждевременному циклу. В рассмотренном примере таким ребром является $j \rightarrow i$ (возможно, существуют и другие). Однако в более сложных случаях несколько ребер могут привести к преждевременной цикличности.

Решение проблемы зацикливания пути Предположим, что у нас есть текущий путь коммивояжера. Все ребра, которые могут создать цикл с текущим путем, уже заблокированы.

На текущей итерации коммивояжер решает выбрать ребро для перемещения $i \rightarrow j$. Чтобы предотвратить возможность зацикливания на следующей итерации, необходимо заблокировать те ребра, которые вместе с ребром $i \rightarrow j$ могут привести к созданию цикла.

Для этого рассмотрим 1ый случай изображенный на рис 2. Новое ребро $i \rightarrow j$ присоединяется к началу существующего пути.

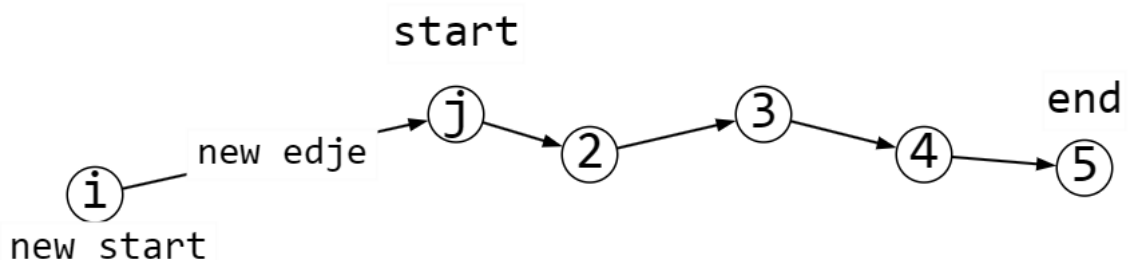


Рис. 2. $i \rightarrow j$ в начале

Все ребра, которые образовывали преждевременный цикл до присоединения города i заблокированы. Тогда осталось заблокировать следующие ребра: $j \rightarrow i$, $2 \rightarrow i$, $3 \rightarrow i$, $4 \rightarrow i$, $5 \rightarrow i$.

Таким образом, чтобы заблокировать нужные ребра в случае, когда ребро $i \rightarrow j$ находится в начале пути, необходимо выполнить следующие операции:

- 1) Найти непрерывный путь, в который входит ребро $i \rightarrow j$ (в данном примере найти путь $i \rightarrow j \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$)
- 2) Пройтись по всем городам CurrCity такого пути до присоединения города i и заблокировать ребра $\text{CurrCity} \rightarrow i$

Случай 2 изображенный на рис. 3. Ребро $i \rightarrow j$ присоединяется в конец существующего пути.

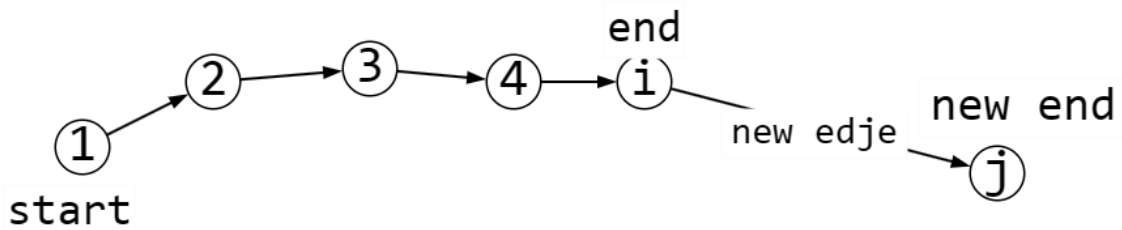


Рис. 3. $i \rightarrow j$ в конце

Осталось заблокировать следующие ребра: $j \rightarrow i$, $j \rightarrow 4$, $j \rightarrow 3$, $j \rightarrow 2$, $j \rightarrow 1$.

Таким образом, чтобы заблокировать нужные ребра в случае, когда ребро $i \rightarrow j$ находится в конце пути, необходимо выполнять следующие операции:

- 1) Найти непрерывный путь, в который входит ребро $i \rightarrow j$ (в данном примере найти путь $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow i \rightarrow j$)
- 2) Пройтись по всем городам CurrCity такого пути до присоединения города j и заблокировать ребра $j \rightarrow \text{CurrCity}$.

Случай 3 изображенный на рис. 4. Ребро $i \rightarrow j$ присоединяется в середину существующего пути.

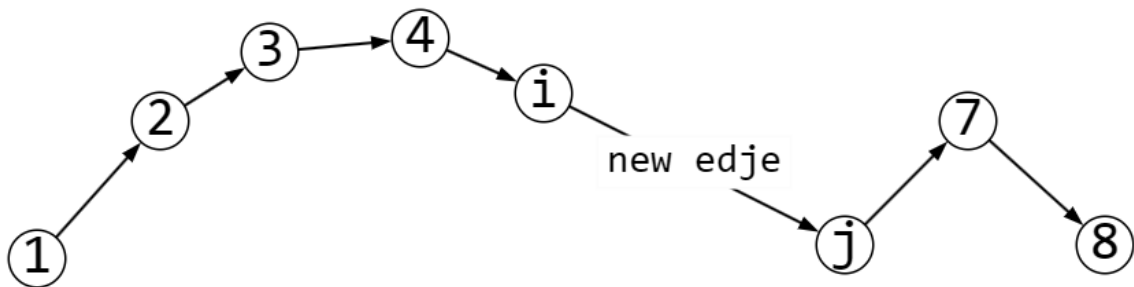


Рис. 4. $i \rightarrow j$ в середине

Осталось заблокировать следующие ребра: $j \rightarrow i$, $j \rightarrow 4$, $j \rightarrow 3$, $j \rightarrow 2$, $j \rightarrow 1$, $7 \rightarrow i$, $7 \rightarrow 4$, $7 \rightarrow 3$, $7 \rightarrow 2$, $7 \rightarrow 1$, $8 \rightarrow i$, $8 \rightarrow 4$, $8 \rightarrow 3$, $8 \rightarrow 2$, $8 \rightarrow 1$.

Таким образом, чтобы заблокировать нужные ребра в случае, когда ребро $i \rightarrow j$ находится в середине пути, необходимо выполнять следующие операции:

1) Найти непрерывный путь, в который входит ребро $i \rightarrow j$ (в данном примере найти путь $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow i \rightarrow j \rightarrow 7 \rightarrow 8$)

2) Пройтись по всем городам CurrCityA у которых позиция больше либо равна позиции j в найденном пути. Для каждого CurrCityA пройти по всем городам CurrCityB у которых позиция меньше либо равна i в найденном пути и заблокировать ребра $\text{CurrCityA} \rightarrow \text{CurrCityB}$.

Ниже приведен псевдокод решающий проблему заикливания коммивояжера. CurrEdge – текущее ребро, которое мы берем (CurrEdge.first – начало ребра, CurrEdge.second – конец ребра), CurrPath – все ребра, которые мы взяли в соответствующей ветке, ContinuousPath – тот самый непрерывный путь, который надо найти во всех 3ех случаях, left – самый левый город в непрерывном пути на каждой итерации, right – самый правый город в непрерывном пути на каждой итерации.

С 1ой по 16ой строки ищем тот самый непрерывный путь. С 17ой по 22ой строки обрабатываем 2ой случай, когда ребро в конце пути. С 23ой по 28ую обрабатываем 1ый случай, когда ребро в начале пути. С 29ой по 42ую обрабатываем 3ий случай, когда ребро в середине. С 30ой по 36ую ищем позицию текущего ребра. С 37ой по 41ую блокируем ребра.

BlockCycle(size, AdjacencyMatrix, CurrEdge, CurrPath)

```

1: ContinuousPath = [CurrEdge.first, CurrEdge.second]
2: left = CurrEdge.first, right = CurrEdge.second, flag = 1
3: while flag do
4:   flag = 0
5:   for edge in CurrPath do
6:     if edge.first == right then
7:       ContinuousPath.append(edge.second)
8:       right = edge.second
9:       flag ++
10:    else if edge.second == left then
11:      ContinuousPath.добавить в начало(edge.first)
12:      left = edge.first
13:      flag ++
14:    end if
15:  end for
16: end while
17: if CurrEdge.first == ContinuousPath[-1] then

```

```

18: ContinuousPath.удалить последний элемент
19: for city in ContinuousPath do
20:     AdjacencyMatrix[CurrEdje.second][city] =  $\infty$ 
21: end for
22: ContinuousPath.append(CurrEdje.first)
23: else if CurrEdje.first == ContinuousPath[0] then
24:     ContinuousPath.удалить первый элемент
25:     for city in ContinuousPath do
26:         AdjacencyMatrix[city][CurrEdje.first] =  $\infty$ 
27:     end for
28:     ContinuousPath.append(CurrEdje.first)
29: else
30:     vertexA = -1, vertexB = -1
31:     for i=0 to ContinuousPath.size()-1 do
32:         if ContinuousPath[i] == CurrEdje.first then
33:             vertexA = i, vertexB = i+1
34:             break
35:         end if
36:     end for
37:     for indexcityA = vertexB to ContinuousPath.size()-1 do
38:         for indexcityB = 0 to vertexA do
39:             AdjacencyMatrix[ContinuousPath[indexcityA]][ContinuousPath[indexcityB]]
=  $\infty$ ;
40:         end for
41:     end for
42: end if

```

Собираем весь алгоритм Ниже представлен псевдокод реализующий алгоритм метода ветвей и границ.

BranchAndBound(size, Matrix, LowerBound, UpperBound, AllowRows, AllowColomns, CurrPath, BestPath)

```

1: LowerBound += ReductionRowAndColumn(size, Matrix, AllowRows, AllowColomns)
2: if LowerBound  $\geq$  UpperBound then
3:     return
4: end if
5: if CurrPath.size() == size-2 then

```

```

6:   Достроить путь до полного
7:   UpperBound = LowerBound
8:   BerstPath = CurrPath
9:   return
10: end if
11: maxPenalty =  $-\infty$ , BestEdgeCoordinates = -1,-1, CurrPenalty = -1,-1
12: for i in AllowRows do
13:   for j in AllowColomns do
14:     if Matrix[i][j] == 0 then
15:       CurrPenalty = PenaltySearch(size, i,j , Matrix, AllowRows, AllowColomns);
16:       if CurrPenalty.first + CurrPenalty.second > maxPenalty then
17:         maxPenalty = CurrPenalty.first + CurrPenalty.second
18:         BestEdjeCoordinates.first = i
19:         BestEdjeCoordinates.second = j
20:       end if
21:     end if
22:   end for
23: end for
24: Matrix[BestEdjeCoordinates.first][BestEdjeCoordinates.second] =  $\infty$ 
25: BranchAndBound(size, LowerBound, UpperBound, Matrix, AllowRows, AllowColomns,
  CurrPath, BestPath)
26: Блокировка строки BestEdjeCoordinates.first
27: Блокировка столбца BestEdjeCoordinates.second
28: BlockCycles(size, BestEdjeCoordinates.first , BestEdjeCoordinates.second , Matrix,
  CurrPath)
29: CurrPath.добавить(BestEdjeCoordinates)
30: BranchAndBound(size, LowerBound, UpperBound, Matrix, TabuRow, TabuColumn,
  CurrPath, BestPath);

```

3. Tabu Search

3.1. Общие сведения

Tabu Search (ТС) — это метаэвристический алгоритм, разработанный для решения сложных комбинаторных задач оптимизации. Основная идея этого метода заключается в использовании памяти для управления процессом поиска и предотвращения возврата к уже исследованным решениям, что позволяет избежать зацик-

ливания на локальных оптимумах.

Рассмотрим несколько отличий метода ветвей и границ и метода поиска с запретами.

1. Эффективность и время выполнения. Метод ветвей и границ не гарантирует, что оптимальное решение будет найдено за приемлемое время. В отличие от него, метод поиска с запретами позволяет настроить количество итераций алгоритма, что обеспечивает выполнение задачи за конечное время. Это делает его более предпочтительным выбором для решения сложных задач, особенно когда время является критическим фактором.

2. Использование памяти. Метод ветвей и границ требует хранения большого количества подзадач. Метод поиска с запретами использует TabuList, который позволяет эффективно контролировать процесс поиска и избегать ненужных возвратов к уже исследованным решениям. Кроме того, размер списка TabuList можно настраивать. Это позволяет адаптировать алгоритм под конкретные требования задачи и оптимизировать использование ресурсов.

3. Оптимальность решения. Метод ветвей и границ является алгоритмом полного перебора, поэтому полученное решение будет оптимальным. Метод поиска с запретами может привести к неоптимальному решению, так как он основан на метаэвристическом подходе. Однако этот метод может быть эффективным для нахождения хороших приближенных решений за разумное время.

Введем некоторые обозначения и сформулируем задачу.

Определение 4 S – множество всех решений, $s \in S$ – допустимое решение. Под решением будем понимать последовательность городов, которые образуют цикл коммивояжера.

Введем функцию f на множестве S : $\forall s \in S, f(s) = \text{длина пути } s$. Тогда задачу коммивояжера можно сформулировать следующим образом. Найти $s_0 : f(s_0) = \min_{s \in S} f(s)$

Определение 5 $N = \{(s, s') \mid s \in S, s' \in S\} \subseteq S \times S$ – окрестностью на S

$N(s) = \{s' \in S \mid (s, s') \in N\}$ – окрестностью решения s

$N^*(s) = \{s' \in N(s) \mid f(s') < f(s)\}$ – оптимальной окрестностью решения s

Определение 6 $s \in S$ – локальный минимум, если $f(s) \leq f(s'), \forall s' \in S$

Окрестности в задаче коммивояжера

Пусть имеем решение $s = \{i_1, \dots, i_k, i_{k+1}, \dots, i_l, i_{l+1}, \dots, i_n\} \in S$

1) Shift

Один из городов i_k ставится на новое место в перестановке см. рис. 5. Окрестность решения s имеет вид $N(s) = \{s' \in S \mid \forall k, l : s' = \{i_1, \dots, i_{k-1}, i_{k+1}, \dots, i_l, i_k, i_{l+1}, \dots, i_n\}\}$.

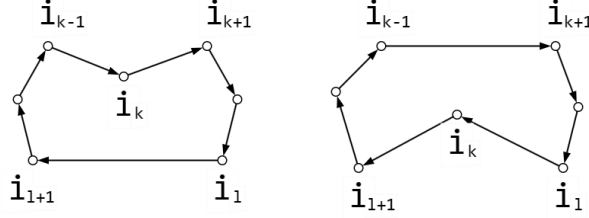


Рис. 5. Shift

Мощность любой такой окрестности $|N(s)| = n(n-2)$.

2) 2-Opt

Выбираются два ребра и меняются их концевые вершины см рис. 6. Окрестность решения s имеет вид $N(s) = \{s' \in S \mid \forall k, l : s' = \{i_1, \dots, i_k, i_l, i_{l-1} \dots, i_{k+1}, i_{l+1}, \dots, i_n\}\}$.

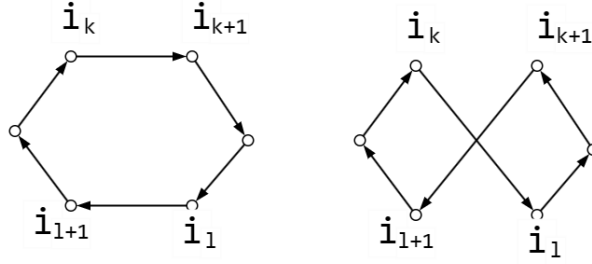


Рис. 6. 2-opt

Окрестность 2-опт хорошо работает на симметричных матрицах. Мощность любой такой окрестности $|N(s)| = \frac{n(n-3)}{2}$.

3) Swap Выбирается некоторое ребро и разворачивается в противоположную сторону. Окрестность решения s имеет вид $N(s) = \{s' \in S \mid \forall k, l : s' = \{i_1, \dots, i_{k-1}, i_{k+1}, i_k, i_{k+2} \dots, i_n\}\}$. Мощность любой такой окрестности $|N(s)| = n$.

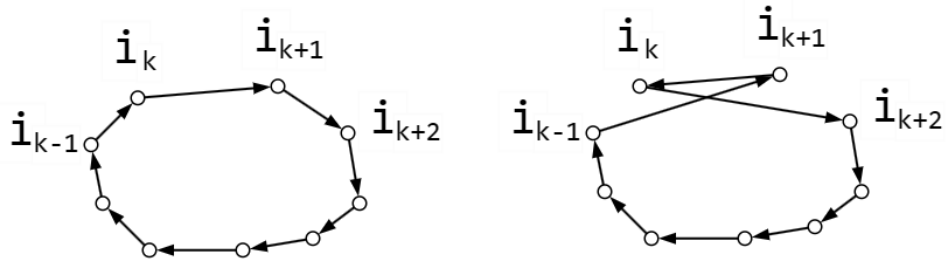


Рис. 7. swap

4) Swap-city Выбираются два города i_k и i_l и они меняются местами см рис. 8. Таким образом окрестность решения s имеет вид $N(s) = \{s' \in S \mid \forall k, l : s' = \{i_1, \dots, i_{k-1}, i_l, i_{k+1}, \dots, i_{l-1}, i_k, i_{l+1}, \dots, i_n\}\}$.

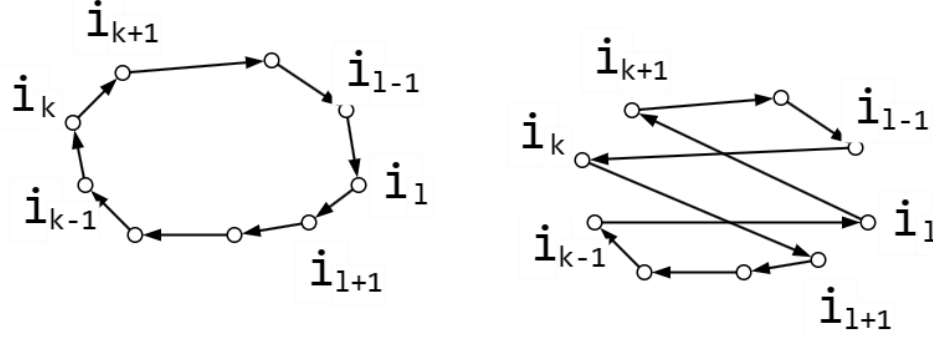


Рис. 8. swapcity

Мощность любой такой окрестности $|N(s)| = \frac{n(n-1)}{2}$.

3.2. Алгоритм локального поиска

Рассмотрим следующую задачу. Имеется функция принимающая дискретные значения. Требуется найти любой локальный минимум. С данной задачей очень хорошо справляется алгоритм локального поиска.

Алгоритм локального поиска — это алгоритм, который используется для решения задач комбинаторной оптимизации. Основная идея алгоритма заключается в том, чтобы начать с некоторого начального решения и затем последовательно улучшать его, переходя от одного локально оптимального решения к другому.

Общая схема алгоритма локального спуска

Шаг 1. Выбрать начальное решение $s \in S$, вычислить $f(s)$.

Шаг 2. Выбрать решение s' из окрестности $N^*(s)$.

Шаг 3. Если решение s' найдено, то $s := s'$ и вернуться на шаг 2 иначе Stop, s — локальный минимум

На шаге 2 выбрать решение s' можно двумя способами. Первый способ — выбрать лучшее решение во всей окрестности $N^*(s)$. Однако этот способ может замедлить работу алгоритма, особенно если мощность множества $|N(s)|$ велика. Придётся перебирать очень много элементов, включая те, которые не подходят. Поэтому лучше выбирать первое попавшееся решение, которое лучше текущего s . Обосновывается это тем, что лучшее решение не может находиться среди решений, которые не попадают в окрестность $N^*(s)$, а находиться рядом с хорошими решениями, то есть в окрестности $N^*(s)$.

Проблема алгоритма локального поиска

Пусть теперь требуется найти глобальный оптимум функции принимающей дискретные значения.

Проблема алгоритма локального поиска заключается в том, что он может застрять в локальном оптимуме и не найти глобальный оптимум. Это происходит потому, что алгоритм выбирает решение из окрестности текущего решения, а не исследует всё пространство решений.

Решение проблемы локального поиска

Один из способов избежать застревания в локальных оптимумах — запускать локальный поиск несколько раз из различных стартовых точек. Однако данные запуски не будут знать о друг друге, поэтому могут упустить глобальный оптимум.

Другой способ — воспользоваться *tabu list*. Он будет хранить информацию о пройденных этапах поиска и позволит алгоритму избегать повторных посещений уже исследованных областей пространства решений. Это поможет предотвратить застревание в локальных оптимумах и повысит шансы на нахождение глобального оптимума.

Tabu list работает следующим образом: когда алгоритм находит новое решение, оно добавляется в список табу. Затем, когда алгоритм выбирает новое направление поиска, он проверяет список табу, чтобы убедиться, что это направление не приведёт к повторному посещению уже исследованной области. Если направление допустимо, алгоритм продолжает поиск; если нет, то алгоритм ищет другое направление. Таким образом, *tabu list* помогает алгоритму исследовать новые области пространства решений и находить более оптимальные решения

3.3. Алгоритм поиск с запретами

Общая схема алгоритма с запретами

Шаг 1. Выбрать начальное решение $s \in S$, вычислить $f(s)$

Шаг 2. Пока не выполнен критерий остановки выполнить:

выбрать наилучшее решение $s' \in N(s) \setminus Tabulist$

$s := s'$

по решению s' обновить *Tabulist*

Шаг 3. Выдать в качестве ответа лучшее из найденных решений.

Построение алгоритма

В алгоритме поиска с запретами будем использовать окрестность *swap-city*.

Рассмотрим следующую вспомогательную задачу. Пусть знаем длину какого-то пути s . Нужно посчитать длину пути нового пути s' после перестановки городов s

индексами i_k и i_l в пути s см. рис. 9

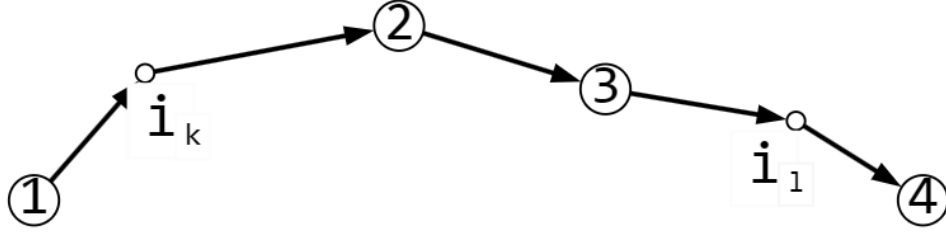


Рис. 9. swapcityexample

Обозначим $Cost$ – текущая стоимость, $path$ – путь коммивояжера для краткости p , i_k, i_l – индексы городов в $path$, которые нужно поменять местами.

Рассмотрим случай когда индексы городов, которые нужно обменять местами не соседние т.е. $|i_k - i_l| > 1$ рис. 9. В таком случае, чтобы получить новую длину пути нужно из старой вычесть стоимости тех ребер, которые входят и выходят из городов с индексами i_k, i_l , а также добавить стоимости новых ребер, которые теперь входят и выходят в i_k, i_l . Таким образом, новую стоимость можно посчитать по формуле

$$Cost_{new} = Cost - \text{стоимость}(\{p[1], p[i_k]\}, \{p[i_k], p[2]\}, \{p[3], p[i_l]\}, \{p[i_l], p[4]\}) \\ + \text{стоимость}(\{p[1], p[i_l]\}, \{p[i_l], p[2]\}, \{p[3], p[i_k]\}, \{p[i_k], p[4]\})$$

Рассмотрим случай когда города, которые нужно обменять соседние рис. 10. Тогда, чтобы получить новую длину пути нужно из старой вычесть стоимости тех ребер, которые входят и выходят в i_k и стоимость того ребра, которое выходит из i_l , а также добавить новые ребра. Таким образом, новую стоимость можно посчитать по формуле

$$Cost_{new} = Cost - \text{стоимость}(\{p[2], p[i_k]\}, \{p[i_k], p[i_l]\}, \{p[i_l], p[3]\}) \\ + \text{стоимость}(\{p[2], p[i_l]\}, \{p[i_l], p[i_k]\}, \{p[i_k], p[3]\})$$

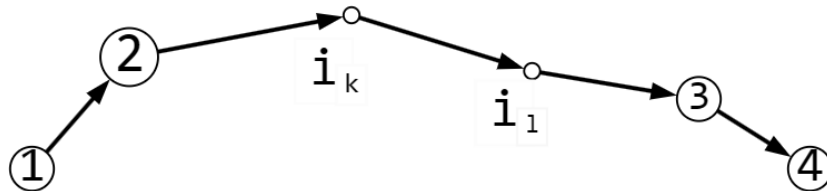


Рис. 10. swapexample1

Ниже представлен псевдокод процедуры пересчета стоимости пути после обмена местами городов. В строках 3–7 обрабатывается случай, когда города, позиции

которых меняются, не являются соседними. В строках 8–10 описывается обработка ситуации, когда города являются соседними. p — это путь коммивояжера, в котором производится перестановка городов, $Cost$ — матрица стоимостей между городами, FI и SI — позиции городов в списке p .

PathCostAfterSwap(size, p, Cost, FI, SI)

```

1: res = p.length
2: p.добавить в конец(0)
3: if  $|FI - SecondSwapIndex| > 1$  then
4:   for index in  $[FI, FI+1, SI, SI+1]$  do
5:     res = res - Cost[p[index - 1]][p[index]]
6:   end for
7:   res = res + Cost[p[SI - 1]][p[FI]] + Cost[p[FI]][p[SI + 1]] + Cost[p[FI - 1]][p[SI]]
   + Cost[p[SI]][p[FI + 1]]
8: else
9:   res = res - Cost[p[FI - 1]][p[FI]] - Cost[p[FI]][p[FI + 1]] - Cost[p[SI]][p[SI + 1]]
10:  res = res + Cost[p[FI - 1]][p[SI]] + Cost[p[SI]][p[FI]] + Cost[p[FI]][p[SI + 1]]
11: end if
12: p.удалить последний элемент
13: Swap(p[FI], p[SI])
14: return res

```

Ниже представлен псевдокод процедуры поиска с запретами для задачи коммивояжера. На каждой итерации внешнего цикла (строки 5–22) осуществляется поиск наилучшей пары индексов для операции swap. В строках 22–24 обновляется список табу: запрещаются перемещения между городами, которые уже были изменены. Строки 25–27 обрабатывают случай, когда алгоритм случайно оказывается в локальном минимуме — в этом случае генерируется новый случайный путь. В строках 29–31 выполняется переход из текущего состояния в наилучшую точку среди ближайших решений.

TabuSearch(size, Matrix, MaxIterations, TabuSize)

```

1: BestPath = Сгенерировать случайный путь коммивояжера, CurrPath = BestPath,
   LocalBestPath = BestPath
2: TabuState = Матрица размера size на size заполненная 0
3: for iteration=0 to MaxIterations do
4:   LocalBestLeftIndex = -1, LocalBestRightIndex = -1

```

```

5:   for i=1 to size-1 do
6:       for j=i+1 to size do
7:           PathCostAfterSwap(size, CurrPath, Matrix, i,j)
8:           if TabuState[CurrPath[i]][CurrPath[j]] > iteration or TabuState[CurrPath[j]][CurrPath[i]] > iteration then
9:               if CurrPath.length < LocalBestPath.length and CurrPath.length < BestPath.length then
10:                  LocalBestPath = CurrPath
11:                  LocalBestLeftIndex = i
12:                  LocalBestRightIndex = j
13:               end if
14:               else if CurrPath.length < LocalBestPath.length then
15:                  LocalBestPath = CurrPath
16:                  LocalBestLeftIndex = i
17:                  LocalBestRightIndex = j
18:               end if
19:               PathCostAfterSwap(size, CurrPath, Matrix, i, j)
20:           end for
21:       end for
22:       if LocalBestLeftIndex != -1 and LocalBestRightIndex != -1 then
23:           TabuState[CurrPath[LocalBestLeftIndex]][CurrPath[LocalBestRightIndex]] = iteration + TabuSize
24:           TabuState[CurrPath[LocalBestRightIndex]][CurrPath[LocalBestLeftIndex]] = iteration + TabuSize
25:       else
26:           CurrPath = Сгенерировать случайный путь коммивояжера
27:           LocalBestPath = CurrPath
28:       end if
29:       CurrPath = LocalBestPath
30:       if CurrPath.length < BestPath.length then
31:           BestPath = CurrPath
32:       end if
33: end for

```

4. Особенности реализации на языке C++

4.1. Метод ветвей и границ

Структуры данных

Для хранения матрицы стоимостей была использована структура двумерный вектор *vector<vector<double>>*.

Вычеркнутые строки и столбцы в процессе алгоритма хранятся в *vector<int>*. *TabuRow*(*TabuColumn*) содержит строки(столбцы), которые вычеркнуты на текущий момент времени.

Путь коммивояжера хранится в структуре *vector<pair<int,int>>*. Координаты ребер хранились с помощью *pair<int,int>*. Также штрафы по строке хранятся с помощью структуры *pair<double,double>*

Функции В ходе реализации поиска с запретами были разработаны следующие функции:

- 1) *GreedyWay* ищет жадный путь коммивояжера.
- 2) *ReductionRows*, *ReductionColumn*, *ReductionRowAndColumn* редуцирование только строк, столбцов, строки и столбцов соответственно.
- 3) *PenaltySearch* ищет штраф не взятия ребра, координаты которого подаются на вход.
- 4) *BlockCycles* блокирует все ребра, которые могут зациклить путь раньше времени. На вход подается текущее ребро и путь.
- 5) *BranchAndBound* метод реализующий метод ветвей границ.
- 6) *CompletePath* данная функция достраивает путь до конца, поскольку после выхода из *BranchAndBound* путь коммивояжера не полный, однако однозначно восстанавливается.

4.2. Поиск с запретами

Структуры данных

Для хранения матрицы стоимостей была использована структура двумерный вектор *vector<vector<double>>*. Это позволяет быстро получать доступ к стоимости перемещения между любыми двумя городами.

Для хранения пути коммивояжера была реализована структура *Tour*. Каждый представитель данной структуры имеет два поля: *path* — последовательность городов, *length* — длина данного пути.

Для хранения запрещенных перестановок мы использовали *TabuList* он был реализован при помощи структуры *vector<vector<double>>*. *TabuList[i][j]* — это номер

итерации начиная с которой можно будет заново менять местами города i, j . То есть, если номер текущей итерации меньше $\text{Tabulist}[i][j]$, то перестановка городов i, j не возможна.

Функции В ходе реализации поиска с запретами были разработаны следующие функции:

- 1) `DataRead` считывает матрицу стоимостей из файла.
- 2) `GenerateRandomPath` генерирует случайный текущий путь коммивояжера (начальная точка, с которой начинается поиск), для этого мы использовали функцию *random_shuffle* из библиотеки `algorithm`, которая перемешивает элементы заданного списка.
- 3) `PathCost` считает стоимость полученного на вход пути.
- 4) `PathCostAfterSwap` пересчитывает стоимость нового пути после перестановки городов i, j , которые функция получает на вход
- 5) `TabuSearch` реализует поиск пути коммивояжера.

5. Особенности реализации на языке C++

5.1. Метод ветвей и границ

Структуры данных

Функции

В ходе реализации поиска с запретами были разработаны следующие функции:

5.2. Поиск с запретами

Структуры данных

Функции

В ходе реализации поиска с запретами были разработаны следующие функции:

Список использованных источников